
Build Your Own AI Coding Agent Harness

An original, source-linked study guide to Vercel Academy's 38-lesson course

An original, source-linked guide to all 38 lessons

26 September 2026

This guide follows Vercel Academy's 11-module learning path. Lesson titles link to the original articles; exact code and version-specific details should be checked there.

Contents

0.1	How to use this guide	2
1	The agent loop and first tools	3
2	Tool contracts and approval	4
3	Instructions and project policy	5
4	Sandbox abstraction and execution backends	6
5	Context, cost, and bounded outputs	7
6	Delegation and subagents	8
7	Cloud sandbox lifecycle	9
8	Human-in-the-loop interaction	10
9	Planning and proof of work	11
10	User-facing surfaces	12
11	Extensibility	13
11.1	A practical implementation order	13
11.2	Blunt feasibility and safety assessment	14
11.3	Source notes	14

Prepared 26 September 2026. This guide explains the course in fresh wording; it is not a word-for-word reproduction. Lesson links point to Vercel’s original articles, which remain the authority for exact code, exercises, and version-specific details.

0.1 How to use this guide

Follow the modules in order. The course is a build-along: start with a tiny tool-calling agent, then grow its tools, policies, execution backends, context strategy, delegation, lifecycle, interaction surfaces, and extension model. For each lesson, read the source, implement the exercise in your own repo, run its checks, and commit the smallest working increment. Do not skip the safety and verification layers just to reach cloud execution faster.

0.1.1 The architecture in one view

The user’s request enters an agent loop. The model sees a carefully composed system prompt and a small set of typed tool descriptions. It chooses tools; the harness validates inputs, applies approval policy, executes against an isolated sandbox adapter, bounds the result, and returns it to the model. The loop continues until the model has enough evidence to answer. Around that core sit project instructions, context pruning, delegation, resumable state, lifecycle timers, user interfaces, and extension points.

Core stack named by the course: TypeScript with Vercel AI SDK, Zod schemas, Bun in the build-along, `just-bash` for an in-memory copy-on-write backend, and Vercel Sandbox as a cloud implementation option. AI Gateway and Vercel Workflow appear in the broader Vercel ecosystem. Treat provider names and APIs as implementation choices; the durable ideas are the interfaces, trust boundaries, and lifecycle contracts.

MODULE 1

The agent loop and first tools

Goal: distinguish a chatbot from an agent by giving the model a controlled way to act, then make its tool set useful without giving it unlimited host access.

1. [From Chat to Agent](#) - Begin with the AI SDK's tool-loop agent. A model with no tools can only respond; adding a typed, callable `read` operation gives it a narrow way to inspect the project. Keep the loop, schemas, and execution in the harness rather than asking the model to simulate tool use.
2. [Your First Tools](#) - Add project search (`grep`) alongside file reading. Tool descriptions are operational guidance: they influence when the model selects a tool, so say what it does, when to use it, and important limits. Validate arguments with a schema and cap results.
3. [Completing the Toolbox](#) - Introduce shell execution only with explicit safety controls. Keep tool definition separate from command execution, classify risky operations, and put approval in the harness. A generic shell is powerful but has a much larger blast radius than read/search tools.

Build checkpoint: the model can read, search, and execute bounded commands; invalid inputs fail clearly; dangerous operations cannot silently run.

MODULE 2

Tool contracts and approval

Goal: make the model's action surface understandable, predictable, and governable.

1. [Descriptions That Work](#) - Treat each tool description and each parameter description as part of the model-facing interface. Explain semantics, defaults, path expectations, limits, and failure conditions. Better descriptions reduce wrong calls; they do not replace server-side validation.
2. [Shell Execution with Safety](#) - Use a tool factory so policy and execution can vary without duplicating every tool. Keep the public schema stable while injecting the sandbox, approval behavior, and output controls behind it.
3. [Approval Gates](#) - Evolve from a simple allow/deny flag to a policy that can distinguish read-only, ordinary writes, and high-impact commands. Approval decisions should be explicit, testable, and attached to the action that needs permission.

Build checkpoint: users can tell what will happen, the harness-not the model-enforces permissions, and approval policy can be changed without rewriting tool implementations.

MODULE 3

Instructions and project policy

Goal: separate stable agent behavior from task-specific and repository-specific guidance.

1. [Structuring Agent Instructions](#) - Organize the system instructions into a role/agency section and a guardrails section. Make tool-first behavior, evidence-based reporting, and limits clear. A structured prompt is easier to review than one long block of mixed policy.
2. [Dynamic Prompt Construction](#) - Compose the final prompt from reusable sections and runtime facts. This gives the harness a controlled place to add the current task, available tools, sandbox capabilities, and project policy without scattering prompt strings throughout the code.
3. [Verification Gates](#) - Put an explicit verification contract in the agent instructions. The agent should distinguish checks it ran from checks it did not run and avoid claiming success merely because it edited files.
4. [Project Context](#) - Load repository instructions such as `AGENTS.md` and expose relevant project commands, conventions, and architecture to the agent. Treat project text as context, not as permission to override the harness's security policy.

Build checkpoint: prompts are assembled predictably, repo-specific guidance is scoped, and completion claims are tied to actual checks.

Sandbox abstraction and execution backends

Goal: decouple tools from where and how code runs, so safety and portability can improve incrementally.

1. [Designing the Interface](#) - Define a small **Sandbox** contract for operations such as reading files and running commands. Optional capabilities-expiry, snapshots, or remote identifiers-should be represented honestly rather than faked by every backend.
2. [Local Implementation](#) - Wrap local filesystem and process APIs behind that contract. This is the easy baseline for development, but it executes with the host user's privileges; it is not a security boundary for untrusted code.
3. [In-Memory Implementation](#) - Use **just-bash** and a virtual, copy-on-write filesystem for cheap, disposable exploration. Account for its virtual mount path and for command behavior that may differ from the host shell.
4. [Cloud Implementation](#) - Map the same contract to a remote VM or sandbox. Remote execution adds provisioning delay, network failure, billing, expiry, and persistence questions. Choose local for trusted iteration, in-memory for safer disposable work, and cloud isolation when the threat model warrants the extra cost.
5. [Lifecycle Hooks](#) - Add hook points such as **afterStart**, **beforeStop**, and **onTimeout** for backend-specific setup and cleanup. Hooks keep the core tool layer stable while lifecycle needs vary by environment.

Build checkpoint: the same agent can run against multiple backends through one interface. Verify writes stay in the intended filesystem and that cleanup runs on both success and failure.

MODULE 5

Context, cost, and bounded outputs

Goal: keep the model's working context useful as tool history accumulates.

1. [The Problem](#) - Instrument token/context growth instead of guessing. Repeated tool calls can fill the conversation with stale details, raising cost and making important facts harder to retrieve.
2. [Pruning Old Results](#) - Remove old tool-call/result pairs while preserving the user's request, relevant assistant reasoning, and recent context. Pruning must keep the message sequence valid; deleting only one half of a tool exchange can corrupt the transcript.
3. [Tool Output Design](#) - Prevent context floods at their source. The course's example budgets are 500 lines for `read`, 50 matches for `grep`, and 5,000 characters for shell output; disclose truncation and offer pagination or narrower queries. Keep a useful tail for command failures where diagnostic output often appears last.
4. [Cache Control](#) - Apply provider-aware prompt caching where supported, especially to repeated stable prefixes. Caching can reduce repeated-input cost, but it is an optimization, not a substitute for shorter prompts or correct context management.

Build checkpoint: measure context size, bound every unbounded output, expose truncation, and test that pruning preserves a coherent tool transcript.

MODULE 6

Delegation and subagents

Goal: split work when doing so improves focus, context isolation, or cost-not simply because parallel agents sound impressive.

1. [Why Delegate](#) - A single agent can accumulate irrelevant context, lose its plan, or hold more capability than a subtask needs. Delegate when the subtask has a clear boundary and the result can be summarized compactly; delegation adds coordination and failure modes.
2. [Explorer Subagent](#) - Create a lower-cost, read-only role for research and codebase exploration. Return findings, paths, and evidence rather than copying an entire secondary transcript into the main context.
3. [Executor Subagent](#) - Use a more capable implementation role only when the task justifies it. A write-capable subagent needs a clear scope, file ownership, tool set, step budget, and a review/verification path.
4. [Task Tool](#) - Route delegation through a single task interface with role-specific constraints and model selection. Treat the ability to create more agents as a privileged capability; constrain spawning, depth, budgets, and returned data.

Build checkpoint: each role has least privilege, bounded work, and a concise result format; the parent remains accountable for integration and verification.

MODULE 7

Cloud sandbox lifecycle

Goal: make remote execution resumable and cost-aware without assuming a VM remains alive forever.

1. [State Machine](#) - Model provisioning, active, hibernating, and hibernated as distinct states. A provider hard-expiry limit differs from the harness's inactivity timeout. Count genuine user/tool activity, not health checks or polling, or idle machines may never hibernate.
2. [Snapshot and Restore](#) - Persist the sandbox state needed to resume work. Design snapshot and restore calls to tolerate retries; repeated requests and partially completed transitions create idempotency hazards. Restoring a filesystem alone does not restore application-level conversation or task state.
3. [Hard-Won Lessons](#) - Plan for stale handles and expiry metadata after reconnects, polling that accidentally resets inactivity, automatic resume loops, and disagreement between the harness's recorded state and the provider's actual state. Reconcile against provider facts and make transitions observable.

Build checkpoint: test timeout boundaries, reconnects, duplicate snapshot requests, expiry, and crash recovery. Decide what data is persisted and how users learn that a sandbox has expired.

MODULE 8

Human-in-the-loop interaction

Goal: ask for decisions in a structured, resumable way and keep approval policy explicit.

1. [Structured Questions](#) - Give the agent a typed `askUser` tool with constrained choices when a decision blocks progress. Make the prompt instruct the model to ask rather than silently guessing on consequential ambiguity; save enough run state to resume after the user answers.
2. [Approval Config](#) - Separate operational modes (for example, interactive versus background) from pluggable event hooks. Configuration is suitable for declarative policy; events support custom integrations. Define how the two layers compose so one cannot accidentally bypass the other.

Build checkpoint: distinguish clarification from permission, pause cleanly, resume with the answer, and record who approved which operation.

MODULE 9

Planning and proof of work

Goal: make multi-step work visible and verify changes using repository evidence.

1. **Todo Tool** - Track task items through pending, in-progress, and completed states. A single active item makes the plan legible; update it as reality changes rather than treating the initial plan as immutable.
2. **Fast Context Understanding** - Search first, then read only files likely to matter. Narrow exploration with paths and focused matches, and parallelize independent reads only after identifying them. This is faster and keeps irrelevant repository data out of context.
3. **Verification Contract** - Define a graduated check sequence such as typecheck, lint, tests, then build. Report which checks passed, failed, or were skipped; only claim what the evidence supports. A successful edit is not proof that the project works.

Build checkpoint: the user can see plan progress, the agent explores selectively, and its final report contains concrete verification evidence.

MODULE 10

User-facing surfaces

Goal: expose one agent runtime through a practical command-line interface and a responsive web chat.

1. [CLI Entry Point](#) - Parse options, select the model and sandbox, initialize the agent, and always clean up in a `finally` path. Keep diagnostics on `stderr` and the answer on `stdout` so the CLI composes with other tools.
2. [Streaming and Tool Rendering](#) - Stream the response and render tool activity as it occurs. Surface status without implying success before the underlying action completes; represent errors and approvals as first-class events.
3. [Web Surface](#) - Reuse the agent logic behind a web UI, adding persistence, resumable streams, and readable tool-result components. The UI needs stable run/message identifiers and should recover from dropped connections rather than lose an in-flight turn.

Build checkpoint: CLI and web entry points invoke the same policy-enforcing runtime; reconnecting a browser does not create duplicate work.

MODULE 11

Extensibility

Goal: let the harness grow through deliberate registration and progressive disclosure.

1. **Skills System** - Keep skill names and short descriptions available to the model, then load full instructions only when a skill is relevant. This progressive-disclosure pattern adds capability without putting every procedure into every prompt. Skills are instructions, not a security boundary.
2. **Custom Tools** - Register new tools through an extension surface instead of hard-coding each one into the agent. Validate schemas and apply the same policy, logging, output limits, and review standards as built-in tools.
3. **Extension Points** - Expose lifecycle events that subscribers can observe, block, modify, or pass through. Define event ordering and chaining rules, and make it clear which hooks may change an action versus only report it.
4. **Durable Workflows** - In-process timers disappear when a function or process ends. A durable workflow engine can sleep across execution boundaries and resume later. Use it for long-lived lifecycle tasks; do not add durable infrastructure to a short-lived local backend that does not need it.

Build checkpoint: extension authors use typed registration and explicit hooks; long-running work survives process restarts only where durability is required.

11.1 A practical implementation order

1. Build the smallest typed read/search agent and observe a full tool cycle.
2. Add a sandbox interface; test local and disposable in-memory execution before remote infrastructure.
3. Introduce command risk tiers, approvals, bounded outputs, and audit events before enabling broad shell access.
4. Add repository instructions, prompt composition, and a verification contract.
5. Measure context growth; add safe pruning and provider-aware caching only after measuring.
6. Add planning and read-only delegation; add write-capable delegation only with scope and review controls.
7. Implement cloud state transitions, snapshots, expiry, and durable coordination as a separate backend concern.
8. Add CLI and web surfaces over the same runtime; then expose carefully versioned extension points.

11.2 Blunt feasibility and safety assessment

The course is a strong harness blueprint, not a turnkey secure autonomous coding service. A local filesystem/process backend is convenient but inherits the machine user's privileges; it should not run arbitrary or untrusted tasks. An in-memory shell reduces risk but is not necessarily a complete isolation boundary. A cloud VM improves isolation only when networking, credentials, persistence, secrets, and teardown are also designed correctly. Likewise, an approval dialog is meaningful only if the enforced decision sits between the model and the executor.

For a credible prototype, first prove a single task end-to-end in a disposable environment: inspect a small repo, make a scoped change, run checks, show the diff, and ask before high-impact actions. Log tool inputs/outputs with redaction, enforce budgets and timeouts, and keep a human-visible stop control. Add autonomy gradually after adversarial tests demonstrate the boundaries-not because a successful happy-path demo looks convincing.

11.3 Source notes

The 38 linked lessons above are Vercel Academy articles in **Build Your Own AI Coding Agent Harness**. Lesson-specific examples, code, exercises, and product behavior can change; check the source article before implementing an API. The guide synthesizes course concepts and is not a substitute for the original sequence.